

LAPORAN PRAKTIKUM PEMROGRAMAN WEB

MODUL 6



**Universitas
Telkom**

Disusun Oleh:

Nama: Dryhus Dzacky Damingtyas.S

NIM: 707012530010

Kelas: D4SIKC-49-04

Mata Kuliah: Pemrograman Web

UNIVERSITAS TELKOM

FAKULTAS ILMU TERAPAN

JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA CERDAS

TAHUN AJARAN 2026

A. Tujuan

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

1. Memahami konsep dasar Pemrograman Berorientasi Objek (OOP) dalam PHP.
2. Menggunakan tipe data object dalam PHP.
3. Membuat dan menggunakan class serta object.
4. Memahami konsep constructor dan destructor.
5. Menerapkan konsep inheritance dalam OOP PHP

B. Alat dan Bahan

Untuk melaksanakan praktikum ini, diperlukan:

1. Komputer dengan sistem operasi Windows, macOS, atau Linux.
2. Web server seperti XAMPP.
3. Editor teks atau IDE seperti VS Code, Sublime Text, atau PHPStorm.
4. Browser web (Google Chrome, Mozilla Firefox, atau lainnya).

C. Langkah-Langkah Praktikum

1. Menyiapkan folder project dan lingkungan kerja.
Project disiapkan di dalam folder Pekan-06 Copy/Pekan-06 dan dijalankan menggunakan XAMPP dengan database MySQL bernama webpro.
2. Membuat struktur program berbasis OOP.
Program disusun menggunakan konsep Object-Oriented Programming dengan memisahkan kode ke dalam beberapa class agar lebih rapi, terstruktur, dan mudah dipelihara.
3. Membuat file bootstrap.php sebagai pemanggil class otomatis.
File ini digunakan untuk mempermudah pemanggilan class dari folder app tanpa harus menulis include atau require satu per satu.
4. Membuat class Database pada folder app/Config.
Class ini berfungsi untuk mengatur koneksi ke database sehingga semua bagian program dapat menggunakan koneksi yang sama.
5. Membuat class UserRepository pada folder app/Repositories.
Class ini digunakan untuk menangani proses akses data user pada database, seperti:
 - mengambil data user,
 - mengecek struktur kolom tabel,
 - menambah data user,
 - mengubah data peserta,
 - menghapus data peserta.
6. Membuat class SessionManager pada folder app/Support.
Class ini berfungsi untuk mengelola session, seperti memulai session, menyimpan data session, mengambil data session, dan menghapus session.
7. Membuat class Response pada folder app/Support.
Class ini digunakan untuk melakukan redirect halaman, misalnya setelah login berhasil atau setelah proses logout.
8. Membuat class RememberMeService pada folder app/Services.

Class ini berfungsi untuk menangani fitur remember me, yaitu menyimpan data login ke cookie dan memulihkan session user dari cookie.

9. Membuat class AuthService pada folder app/Services.

Class ini digunakan untuk menangani proses autentikasi, seperti:

- login user,
- registrasi user,
- logout user,
- pengecekan apakah user sudah login atau belum.

10. Membuat class ParticipantPhotoService pada folder app/Services.

Class ini dibuat untuk mengatur proses upload dan hapus file foto peserta, termasuk validasi format gambar dan ukuran file.

11. Membuat class ParticipantService pada folder app/Services.

Class ini digunakan untuk menangani logika utama fitur kelola peserta, seperti:

- menambah data peserta,
- mengedit data peserta,
- menghapus data peserta,
- mengambil daftar peserta dari database.

12. Menghubungkan halaman register.php dengan class OOP.

Halaman register dibuat agar input dari form diproses oleh AuthService, kemudian data user disimpan ke database melalui UserRepository.

13. Menghubungkan halaman auth.php dengan class OOP.

Halaman login dibuat agar proses login dilakukan oleh AuthService, termasuk validasi email, password, role, dan fitur remember me.

14. Menghubungkan halaman kelolapeserta.php dengan class OOP.

Halaman ini menggunakan ParticipantService untuk mengelola data peserta, sedangkan AuthService digunakan untuk memastikan hanya user yang sudah login yang dapat mengakses halaman.

15. Menghubungkan halaman logout.php dengan class OOP.

Halaman logout dibuat agar proses keluar dari sistem dilakukan melalui AuthService dengan menghapus session dan cookie login.

16. Menambahkan komentar pada setiap file program.

Setiap bagian kode diberi komentar untuk menjelaskan fungsi dari baris atau blok program agar lebih mudah dipahami.

17. Melakukan pengujian program.

Pengujian dilakukan pada fitur-fitur utama, yaitu:

- registrasi akun,
- login user,
- logout user,
- tambah peserta,

- edit peserta,
- hapus peserta,
- upload foto peserta.

18. Memeriksa sintaks program.

Seluruh file PHP diperiksa menggunakan perintah `php -l` untuk memastikan tidak ada kesalahan sintaks.

19. Menganalisis hasil praktikum.

Dari hasil praktikum, penerapan OOP membuat program menjadi lebih modular, lebih rapi, dan lebih mudah dikembangkan dibandingkan penulisan kode secara prosedural.

D. Hasil dan Dokumentasi Program

```
<?php

declare(strict_types=1);

namespace App\Config;

use mysqli;
use RuntimeException;

// Class ini bertugas menyediakan satu koneksi database yang bisa dipakai bersama.
final class Database
{
    // Menyimpan instance koneksi agar tidak dibuat berulang-ulang.
    private static ?mysqli $connection = null;

    // Method statis untuk mengambil koneksi database aktif.
    public static function getConnection(): mysqli
    {
        // Jika koneksi sudah pernah dibuat, langsung gunakan kembali.
        if (self::$connection instanceof mysqli) {
            return self::$connection;
        }

        // Membuat koneksi ke database MySQL.
        $connection = mysqli_connect("localhost", "root", "", "webpro");

        // Hentikan proses jika koneksi gagal.
        if (!$connection instanceof mysqli) {
            throw new RuntimeException("database tidak terhubung");
        }

        // Mengatur charset agar mendukung karakter UTF-8.
        mysqli_set_charset($connection, "utf8mb4");
        // Menyimpan koneksi ke property statis.
        self::$connection = $connection;
    }
}
```

```

        // Mengembalikan koneksi yang sudah siap dipakai.
        return self::$connection;
    }
}

```

INI ADALAH database.php

```

<?php

declare(strict_types=1);

namespace App\Repositories;

use mysqli;

// Repository ini menangani seluruh operasi data pada tabel user.
final class UserRepository
{
    // Nama tabel utama yang dipakai aplikasi.
    private string $table = "user";

    // Menerima object koneksi database dari luar class.
    public function __construct(private mysqli $connection)
    {
    }

    // Mengambil daftar semua kolom yang ada pada tabel user.
    public function getColumns(): array
    {
        $columns = [];
        $query = mysqli_query($this->connection, "SHOW COLUMNS FROM {$this->table}");

        if ($query) {
            while ($column = mysqli_fetch_assoc($query)) {
                $columns[] = $column["Field"];
            }
        }

        return $columns;
    }

    // Mengecek apakah suatu kolom tersedia di tabel user.
    public function hasColumn(string $column): bool
    {
        return in_array($column, $this->getColumns(), true);
    }

    // Memastikan kolom foto tersedia sebelum fitur upload digunakan.
    public function ensureFotoColumn(): void
    {
        if ($this->hasColumn("foto")) {
            return;
        }
    }
}

```

```

    }

    mysqli_query($this->connection, "ALTER TABLE {$this->table} ADD COLUMN
foto VARCHAR(255) NULL");
}

// Mengambil satu user berdasarkan email.
public function findByEmail(string $email): ?array
{
    $statement = mysqli_prepare($this->connection, "SELECT * FROM {$this-
>table} WHERE email = ? LIMIT 1");
    if (!$statement) {
        return null;
    }

    mysqli_stmt_bind_param($statement, "s", $email);
    mysqli_stmt_execute($statement);
    $result = mysqli_stmt_get_result($statement);
    $user = $result ? mysqli_fetch_assoc($result) : null;
    mysqli_stmt_close($statement);

    return $user ?: null;
}

// Mengambil user berdasarkan kombinasi email, password, dan role.
public function findByCredentials(string $email, string $passwordHash,
string $role): ?array
{
    if ($this->hasColumn("role")) {
        $statement = mysqli_prepare($this->connection, "SELECT * FROM
{$this->table} WHERE email = ? AND password = ? AND role = ? LIMIT 1");
        if (!$statement) {
            return null;
        }

        mysqli_stmt_bind_param($statement, "sss", $email, $passwordHash,
$role);
    } else {
        $statement = mysqli_prepare($this->connection, "SELECT * FROM
{$this->table} WHERE email = ? AND password = ? LIMIT 1");
        if (!$statement) {
            return null;
        }

        mysqli_stmt_bind_param($statement, "ss", $email, $passwordHash);
    }

    mysqli_stmt_execute($statement);
    $result = mysqli_stmt_get_result($statement);
    $user = $result ? mysqli_fetch_assoc($result) : null;
    mysqli_stmt_close($statement);

    return $user ?: null;
}

```

```

    }

    // Menambahkan akun user baru ke database.
    public function createUser(string $username, string $email, string
$passwordHash, string $role): array
    {
        if ($this->hasColumn("role")) {
            $statement = mysqli_prepare($this->connection, "INSERT INTO
{$this->table} (username, email, password, role) VALUES (?, ?, ?, ?)");
            if (!$statement) {
                return [false, "Gagal menyiapkan query: " .
mysqli_error($this->connection)];
            }

            mysqli_stmt_bind_param($statement, "ssss", $username, $email,
$passwordHash, $role);
        } else {
            $statement = mysqli_prepare($this->connection, "INSERT INTO
{$this->table} (username, email, password) VALUES (?, ?, ?)");
            if (!$statement) {
                return [false, "Gagal menyiapkan query: " .
mysqli_error($this->connection)];
            }

            mysqli_stmt_bind_param($statement, "sss", $username, $email,
$passwordHash);
        }

        $success = mysqli_stmt_execute($statement);
        $message = $success ? "Registrasi berhasil." : "Registrasi gagal: " .
mysqli_stmt_error($statement);
        mysqli_stmt_close($statement);

        return [$success, $message];
    }

    // Menentukan nama kolom-kolom yang dipakai untuk fitur peserta.
    public function getParticipantSchema(): array
    {
        $columns = $this->getColumns();

        return [
            "id" => in_array("id", $columns, true) ? "id" :
(in_array("id_user", $columns, true) ? "id_user" : null),
            "nama" => in_array("nama", $columns, true) ? "nama" :
(in_array("username", $columns, true) ? "username" : null),
            "umur" => in_array("umur", $columns, true) ? "umur" :
(in_array("usia", $columns, true) ? "usia" : null),
            "jenis_kelamin" => in_array("jenis_kelamin", $columns, true) ?
"jenis_kelamin" : (in_array("jenisKelamin", $columns, true) ? "jenisKelamin" :
null),
            "pelatihan" => in_array("pelatihan", $columns, true) ? "pelatihan"
: null,

```

```

        "foto" => in_array("foto", $columns, true) ? "foto" : null,
    ];
}

// Mengecek apakah struktur tabel sudah cukup untuk mengelola peserta.
public function canManageParticipants(): bool
{
    $schema = $this->getParticipantSchema();

    return $schema["id"] && $schema["nama"] && $schema["umur"] &&
    $schema["jenis_kelamin"] && $schema["pelatihan"];
}

// Mengambil seluruh data peserta yang siap ditampilkan.
public function getParticipants(): array
{
    $schema = $this->getParticipantSchema();
    if (!$this->canManageParticipants()) {
        return [];
    }

    $selectFoto = $schema["foto"] ? ", {$schema["foto"]} AS foto" : ", '
AS foto";
    $sql = "SELECT
        {$schema["id"]} AS id,
        {$schema["nama"]} AS nama,
        {$schema["umur"]} AS umur,
        {$schema["jenis_kelamin"]} AS jenis_kelamin,
        {$schema["pelatihan"]} AS pelatihan
        {$selectFoto}
    FROM {$this->table}
    WHERE {$schema["nama"]} IS NOT NULL AND TRIM({$schema["nama"]}) <>
    ''

        AND {$schema["umur"]} IS NOT NULL
        AND {$schema["jenis_kelamin"]} IS NOT NULL AND
    TRIM({$schema["jenis_kelamin"]}) <> ''
        AND {$schema["pelatihan"]} IS NOT NULL AND
    TRIM({$schema["pelatihan"]}) <> ''
    ORDER BY {$schema["id"]} ASC";

    $rows = [];
    $query = mysqli_query($this->connection, $sql);
    if ($query) {
        while ($row = mysqli_fetch_assoc($query)) {
            $rows[] = $row;
        }
    }

    return $rows;
}

// Menyimpan data peserta baru ke database.
public function createParticipant(array $data): array

```



```

{
    $schema = $this->getParticipantSchema();
    $columns = [$schema["nama"], $schema["umur"],
$schema["jenis_kelamin"], $schema["pelatihan"]];
    $values = [$data["nama"], (int)$data["umur"], $data["jenis_kelamin"],
$data["pelatihan"]];
    $types = "siss";

    if ($schema["foto"]) {
        $columns[] = $schema["foto"];
        $values[] = $data["foto"];
        $types .= "s";
    }

    $statement = mysqli_prepare(
        $this->connection,
        "INSERT INTO {$this->table} (" . implode(", ", $columns) . ")
VALUES (" . implode(", ", array_fill(0, count($columns), "?")) . ")"
    );

    if (!$statement) {
        return [false, "Prepare query gagal: " . mysqli_error($this-
>connection)];
    }

    $this->bindDynamic($statement, $types, $values);
    $success = mysqli_stmt_execute($statement);
    $message = $success ? "Data peserta berhasil ditambahkan." : "Gagal
menyimpan data: " . mysqli_stmt_error($statement);
    mysqli_stmt_close($statement);

    return [$success, $message];
}

// Memperbarui data peserta yang sudah ada.
public function updateParticipant(array $data): array
{
    $schema = $this->getParticipantSchema();
    $sql = "UPDATE {$this->table} SET {$schema["nama"]} = ?,
{$schema["umur"]} = ?, {$schema["jenis_kelamin"]} = ?, {$schema["pelatihan"]}
= ?";
    $values = [$data["nama"], (int)$data["umur"], $data["jenis_kelamin"],
$data["pelatihan"]];
    $types = "siss";

    if ($schema["foto"]) {
        $sql .= ", {$schema["foto"]} = ?";
        $values[] = $data["foto"];
        $types .= "s";
    }

    $sql .= " WHERE {$schema["id"]} = ? LIMIT 1";
    $values[] = (int)$data["id"];

```

```

        $types .= "i";

        $statement = mysqli_prepare($this->connection, $sql);
        if (!$statement) {
            return [false, "Prepare query edit gagal: " . mysqli_error($this->connection)];
        }

        $this->bindDynamic($statement, $types, $values);
        $success = mysqli_stmt_execute($statement);
        $message = $success ? "Data peserta berhasil diperbarui." : "Gagal memperbarui data: " . mysqli_stmt_error($statement);
        mysqli_stmt_close($statement);

        return [$success, $message];
    }

    // Menghapus data peserta berdasarkan id.
    public function deleteParticipant(int $id): array
    {
        $schema = $this->getParticipantSchema();
        $statement = mysqli_prepare($this->connection, "DELETE FROM {$this->table} WHERE {$schema["id"]} = ? LIMIT 1");

        if (!$statement) {
            return [false, "Prepare query hapus gagal: " . mysqli_error($this->connection)];
        }

        mysqli_stmt_bind_param($statement, "i", $id);
        $success = mysqli_stmt_execute($statement);
        $message = $success ? "Data peserta berhasil dihapus." : "Gagal menghapus data: " . mysqli_stmt_error($statement);
        mysqli_stmt_close($statement);

        return [$success, $message];
    }

    // Helper untuk bind parameter dinamis pada prepared statement.
    private function bindDynamic(\mysqli_stmt $statement, string $types, array &$values): void
    {
        $params = [$statement, $types];

        foreach ($values as $key => $value) {
            $params[] = &$values[$key];
        }

        call_user_func_array("mysqli_stmt_bind_param", $params);
    }
}

```

INI ADALAH UserRepositoryi.php

```

<?php

declare(strict_types=1);

namespace App\Services;

use App\Repositories\UserRepository;
use App\Support\Response;
use App\Support\SessionManager;

// Service ini menangani alur autentikasi seperti login, register, dan logout.
final class AuthService
{
    // Constructor menerima object session, repository, dan remember me
    // service.
    public function __construct(
        private SessionManager $session,
        private UserRepository $userRepository,
        private RememberMeService $rememberMe
    ) {
    }

    // Mengambil email user yang sedang login dari session.
    public function currentUser(): ?string
    {
        $user = $this->session->get("user");

        return $user === null || $user === "" ? null : (string)$user;
    }

    // Mengambil nama user yang sedang login untuk ditampilkan di halaman.
    public function username(): string
    {
        // Ambil username yang tersimpan di session.
        $username = trim((string)$this->session->get("username", ""));
        // Ambil email user yang sedang login sebagai acuan fallback.
        $currentEmail = trim((string)$this->session->get("user", ""));

        // Jika session username sudah ada dan bukan email, langsung gunakan.
        if ($username !== "" && $username !== $currentEmail) {
            return $username;
        }

        // Ambil ulang data user dari database untuk memastikan nama yang
        // tampil adalah nama register.
        if ($currentEmail !== "") {
            $user = $this->userRepository->findByEmail($currentEmail);

            if ($user !== null) {
                $resolvedUsername = trim((string)($user["username"] ??
($user["nama"] ?? "")));
            }
        }
    }
}

```

```

        // Jika nama ditemukan, perbarui session lalu tampilkan nama
        tersebut.
        if ($resolvedUsername !== "") {
            $this->session->set("username", $resolvedUsername);

            return $resolvedUsername;
        }
    }

    // Jika nama tetap tidak ditemukan, gunakan nilai session lama.
    return $username;
}

// Mencoba memulihkan login user dari cookie remember me.
public function restoreUserFromCookie(): bool
{
    if ($this->currentUser() !== null) {
        return true;
    }

    $user = $this->rememberMe->restore();
    if ($user === null) {
        return false;
    }

    $this->storeUserSession($user);

    return true;
}

// Menangani proses login user.
public function login(string $email, string $password, string $role, bool
$rememberMe): array
{
    if ($email === "" || $password === "" || $role === "") {
        return [false, "Email, password, dan role wajib diisi."];
    }

    $user = $this->userRepository->findByCredentials($email,
md5($password), $role);
    if ($user === null) {
        return [false, "Email, password, atau role tidak cocok."];
    }

    $this->storeUserSession($user);

    if ($rememberMe) {
        $this->rememberMe->set($user);
    } else {
        $this->rememberMe->clear();
    }
}

```

```

        return [true, ""];
    }

    // Menangani proses registrasi akun baru.
    public function register(string $username, string $email, string
$password, string $role): array
    {
        if ($username === "" || $email === "" || $password === "") {
            return [false, "Username, email, dan password wajib diisi."];
        }

        return $this->userRepository->createUser($username, $email,
md5($password), $role);
    }

    // Menjalankan proses logout user dari session dan cookie.
    public function logout(): void
    {
        $this->rememberMe->clear();
        $this->session->destroy();
    }

    // Memastikan user wajib login sebelum membuka halaman tertentu.
    public function requireLogin(string $redirectTo = "auth.php"): void
    {
        $this->restoreUserFromCookie();

        if ($this->currentUser() === null) {
            Response::redirect($redirectTo);
        }
    }

    // Menyimpan data penting user ke session setelah login berhasil.
    private function storeUserSession(array $user): void
    {
        $this->session->set("user", $user["email"] ?? "");
        $this->session->set("role", $user["role"] ?? "");
        $this->session->set("username", $user["username"] ?? ($user["nama"] ??
""));
    }
}

```

INI ADALAH AuthService.php

```

<?php

declare(strict_types=1);

namespace App\Services;

// Service ini menangani upload dan hapus file foto peserta.
final class ParticipantPhotoService
{
    // Menerima folder upload sebagai lokasi penyimpanan file.
}

```

```

public function __construct(private string $uploadDirectory)
{
}

// Mengunggah file foto peserta dan mengembalikan status hasil upload.
public function upload(array $file, bool $required = true): array
{
    // Jika file tidak dipilih, kembalikan error saat upload wajib.
    if (($file["error"] ?? UPLOAD_ERR_NO_FILE) === UPLOAD_ERR_NO_FILE) {
        return $required ? [false, "Foto peserta wajib diupload.", ""] :
[true, "", ""];
    }

    // Hentikan proses jika upload file gagal.
    if (($file["error"] ?? UPLOAD_ERR_NO_FILE) !== UPLOAD_ERR_OK) {
        return [false, "Upload gambar gagal.", ""];
    }

    // Pastikan file yang diupload benar-benar berupa gambar.
    if (@getimagesize($file["tmp_name"]) === false) {
        return [false, "File yang diupload harus berupa gambar.", ""];
    }

    // Batasi ekstensi file gambar yang boleh diupload.
    $validExtensions = ["jpg", "jpeg", "png", "gif", "webp"];
    $extension = strtolower(pathinfo($file["name"] ?? "",
PATHINFO_EXTENSION));
    if (!in_array($extension, $validExtensions, true)) {
        return [false, "Format gambar harus jpg, jpeg, png, gif, atau
webp.", ""];
    }

    // Batasi ukuran file maksimal 2 MB.
    if (($file["size"] ?? 0) > 2 * 1024 * 1024) {
        return [false, "Ukuran gambar maksimal 2 MB.", ""];
    }

    // Buat folder upload jika belum tersedia.
    if (!is_dir($this->uploadDirectory) && !mkdir($this->uploadDirectory,
0777, true)) {
        return [false, "Folder uploads tidak bisa dibuat.", ""];
    }

    // Buat nama file unik agar tidak bentrok dengan file lain.
    $newFilename = uniqid("peserta_", true) . "." . $extension;
    $targetPath = $this->uploadDirectory . DIRECTORY_SEPARATOR .
$newFilename;

    // Pindahkan file dari folder sementara ke folder upload.
    if (!move_uploaded_file($file["tmp_name"], $targetPath)) {
        return [false, "Gagal memindahkan gambar ke folder uploads.", ""];
    }
}

```

```

        // Kembalikan status sukses beserta nama file baru.
        return [true, "", $newFilename];
    }

    // Menghapus file foto peserta jika file-nya ada.
    public function delete(string $filename): void
    {
        if ($filename === "") {
            return;
        }

        $filePath = $this->uploadDirectory . DIRECTORY_SEPARATOR . $filename;
        if (is_file($filePath)) {
            unlink($filePath);
        }
    }
}

```

INI ADALAH ParticipantPhotoService

```

<?php

declare(strict_types=1);

namespace App\Services;

use App\Repositories\UserRepository;

// Service ini menangani proses bisnis untuk fitur kelola peserta.
final class ParticipantService
{
    // Constructor menerima repository data dan service upload foto.
    public function __construct(
        private UserRepository $userRepository,
        private ParticipantPhotoService $photoService
    ) {
    }

    // Memastikan struktur tabel peserta siap digunakan.
    public function canManageParticipants(): bool
    {
        $this->userRepository->ensureFotoColumn();

        return $this->userRepository->canManageParticipants();
    }

    // Mengambil seluruh data peserta dari repository.
    public function getParticipants(): array
    {
        return $this->userRepository->getParticipants();
    }

    // Menangani request form peserta untuk tambah, edit, atau hapus.
    public function handleForm(array $post, array $files): array
    {
    }
}

```

```

{
    // Ambil aksi form, default-nya tambah.
    $action = $post["aksi"] ?? "tambah";

    // Jika aksi hapus, langsung proses penghapusan data.
    if ($action === "hapus") {
        return $this->deleteParticipant((int)($post["id"] ?? 0),
trim((string)($post["foto_lama"] ?? "")));
    }

    // Siapkan data peserta dari input form.
    $payload = [
        "id" => (int)($post["id"] ?? 0),
        "nama" => trim((string)($post["nama"] ?? "")),
        "umur" => trim((string)($post["umur"] ?? "")),
        "jenis_kelamin" => trim((string)($post["jenis_kelamin"] ?? "")),
        "pelatihan" => trim((string)($post["pelatihan"] ?? "")),
        "foto_lama" => trim((string)($post["foto_lama"] ?? "")),
    ];

    // Validasi field wajib sebelum proses simpan data.
    if ($payload["nama"] === "" || $payload["umur"] === "" ||
$payload["jenis_kelamin"] === "" || $payload["pelatihan"] === "") {
        return [false, "Nama, umur, jenis kelamin, dan pelatihan wajib
diisi.", false];
    }

    // Cek apakah struktur tabel mendukung fitur peserta.
    if (!$this->userRepository->canManageParticipants()) {
        return [false, "Struktur tabel user belum lengkap untuk kelola
peserta.", false];
    }

    // Tandai apakah proses saat ini adalah tambah data baru.
    $isCreate = $action === "tambah";
    // Jalankan proses upload foto peserta.
    [$uploadSuccess, $uploadMessage, $photoName] = $this->photoService-
>upload($files["foto"] ?? [], $isCreate);

    // Hentikan proses jika upload gagal.
    if (!$uploadSuccess) {
        return [false, $uploadMessage, false];
    }

    // Gunakan foto baru jika ada, atau pertahankan foto lama saat edit.
    $payload["foto"] = $photoName !== "" ? $photoName :
$payload["foto_lama"];
    // Simpan data ke repository sesuai aksi tambah atau edit.
    $result = $isCreate
        ? $this->userRepository->createParticipant($payload)
        : $this->userRepository->updateParticipant($payload);

    // Hapus file baru jika penyimpanan database gagal.

```



```

        if (!$result[0] && $photoName !== "") {
            $this->photoService->delete($photoName);
        }

        // Hapus foto lama jika edit berhasil dan user mengganti fotonya.
        if ($result[0] && !$isCreate && $photoName !== "" &&
        $payload["foto_lama"] !== "" && $payload["foto_lama"] !== $photoName) {
            $this->photoService->delete($payload["foto_lama"]);
        }

        // Kembalikan status proses, pesan, dan penanda redirect.
        return [$result[0], $result[1], $result[0]];
    }

    // Menghapus data peserta beserta foto lamanya.
    private function deleteParticipant(int $id, string $oldPhoto): array
    {
        $result = $this->userRepository->deleteParticipant($id);

        if ($result[0]) {
            $this->photoService->delete($oldPhoto);
        }

        return [$result[0], $result[1], $result[0]];
    }
}

```

INI ADALAH ParticipantService.php

```

<?php

declare(strict_types=1);

namespace App\Services;

use App\Repositories\UserRepository;

// Service ini menangani fitur remember me berbasis cookie.
final class RememberMeService
{
    // Nama cookie yang dipakai untuk menyimpan data remember me.
    private const COOKIE_NAME = "inkluskill_remember";
    // Lama aktif cookie remember me dalam hari.
    private const COOKIE_DAYS = 7;
    // Secret key untuk membuat signature cookie.
    private const SECRET_KEY = "inkluskill_pekan5_secret_key";

    // Constructor menerima repository user untuk validasi data dari cookie.
    public function __construct(private UserRepository $userRepository)
    {
    }

    // Menyimpan cookie remember me saat user memilih tetap login.
    public function set(array $user): void
    {
    }
}

```

```

{
    $identifier = $this->getIdentifier($user);
    if ($identifier === "") {
        return;
    }

    $expiresAt = time() + (self::COOKIE_DAYS * 24 * 60 * 60);
    $payload = json_encode([
        "identifier" => $identifier,
        "expires_at" => $expiresAt,
        "signature" => $this->buildSignature($user, $expiresAt),
    ]);

    if ($payload === false) {
        return;
    }

    setcookie(
        self::COOKIE_NAME,
        base64_encode($payload),
        [
            "expires" => $expiresAt,
            "path" => "/",
            "httponly" => true,
            "samesite" => "Lax",
        ]
    );
}

// Menghapus cookie remember me dari browser.
public function clear(): void
{
    setcookie(
        self::COOKIE_NAME,
        "",
        [
            "expires" => time() - 3600,
            "path" => "/",
            "httponly" => true,
            "samesite" => "Lax",
        ]
    );
}

// Memulihkan data user dari cookie remember me yang masih valid.
public function restore(): ?array
{
    if (empty($_COOKIE[self::COOKIE_NAME])) {
        return null;
    }

    $decoded = base64_decode($_COOKIE[self::COOKIE_NAME], true);
    if ($decoded === false) {

```

```

        $this->clear();

        return null;
    }

    $data = json_decode($decoded, true);
    if (!is_array($data)) {
        $this->clear();

        return null;
    }

    $identifier = trim((string)($data["identifier"] ?? ""));
    $expiresAt = (int)($data["expires_at"] ?? 0);
    $signature = (string)($data["signature"] ?? "");

    if ($identifier === "" || $expiresAt < time() || $signature === "") {
        $this->clear();

        return null;
    }

    $user = $this->userRepository->findByEmail($identifier);
    if ($user === null) {
        $this->clear();

        return null;
    }

    if (!hash_equals($this->buildSignature($user, $expiresAt),
$signature)) {
        $this->clear();

        return null;
    }

    return $user;
}

// Mengambil identifier user yang dipakai di cookie, yaitu email.
private function getIdentifier(array $user): string
{
    return (string)($user["email"] ?? "");
}

// Membuat signature untuk memastikan cookie tidak dimanipulasi.
private function buildSignature(array $user, int $expiresAt): string
{
    $identifier = $this->getIdentifier($user);
    $passwordHash = (string)($user["password"] ?? "");
    $role = (string)($user["role"] ?? "");

```

```

        return hash_hmac("sha256", $identifier . "|" . $role . "|" .
$expiresAt, self::SECRET_KEY . $passwordHash);
    }
}

```

INI ADALAH RememberMeService.php

```

<?php

declare(strict_types=1);

namespace App\Support;

// Helper sederhana untuk kebutuhan response HTTP.
final class Response
{
    // Mengarahkan user ke halaman lain lalu menghentikan proses script.
    public static function redirect(string $location): never
    {
        header("Location: " . $location);
        exit();
    }
}

```

INI ADALAH Response.php

```

<?php

declare(strict_types=1);

namespace App\Support;

// Class helper untuk mengelola session aplikasi.
final class SessionManager
{
    // Memastikan session aktif sebelum digunakan.
    public function start(): void
    {
        if (session_status() === PHP_SESSION_NONE) {
            session_start();
        }
    }

    // Mengambil nilai session berdasarkan key tertentu.
    public function get(string $key, mixed $default = null): mixed
    {
        $this->start();

        return $_SESSION[$key] ?? $default;
    }

    // Menyimpan nilai baru ke dalam session.
    public function set(string $key, mixed $value): void
    {
        $this->start();
    }
}

```

```

        $_SESSION[$key] = $value;
    }

    // Menghapus satu data session berdasarkan key.
    public function remove(string $key): void
    {
        $this->start();
        unset($_SESSION[$key]);
    }

    // Mengambil data session lalu langsung menghapusnya.
    public function pull(string $key, mixed $default = null): mixed
    {
        $value = $this->get($key, $default);
        $this->remove($key);

        return $value;
    }

    // Menghapus seluruh data session dan cookie session.
    public function destroy(): void
    {
        $this->start();
        $_SESSION = [];

        if (ini_get("session.use_cookies")) {
            $params = session_get_cookie_params();
            setcookie(
                session_name(),
                "",
                time() - 42000,
                $params["path"],
                $params["domain"],
                $params["secure"],
                $params["httponly"]
            );
        }

        session_destroy();
    }
}

```

INI ADALAH SessionManager.php

```

<?php

declare(strict_types=1);

// Autoloader ini otomatis memuat class sesuai namespace App\..
spl_autoload_register(static function (string $class): void {
    // Prefix namespace yang dipakai dalam aplikasi.
    $prefix = "App\\";
    // Folder dasar tempat seluruh class disimpan.
    $baseDir = __DIR__ . DIRECTORY_SEPARATOR;

```

```

    // Abaikan class yang bukan bagian dari namespace App\
    if (strncmp($class, $prefix, strlen($prefix)) !== 0) {
        return;
    }

    // Ambil nama class relatif tanpa prefix App\
    $relativeClass = substr($class, strlen($prefix));
    // Bentuk lokasi file class berdasarkan namespace.
    $file = $baseDir . str_replace("\\", DIRECTORY_SEPARATOR, $relativeClass)
    . ".php";

    // Jika file class ditemukan, muat file tersebut.
    if (is_file($file)) {
        require_once $file;
    }
});

// Menetapkan timezone aplikasi ke Asia/Jakarta.
date_default_timezone_set("Asia/Jakarta");

```

INI ADALAH Bootstrap.php

```

<?php

declare(strict_types=1);

// Memuat autoloader class OOP aplikasi.
require_once __DIR__ . "/app/bootstrap.php";

use App\Config\Database;
use App\Repositories\UserRepository;
use App\Services\AuthService;
use App\Services\RememberMeService;
use App\Support\Response;
use App\Support\SessionManager;

// Menyiapkan session manager untuk kebutuhan login.
$session = new SessionManager();
// Memastikan session aktif.
$session->start();
// Menyiapkan repository data user.
$userRepository = new UserRepository(Database::getConnection());
// Menyiapkan service auth yang menangani login dan logout.
$authService = new AuthService($session, $userRepository, new
RememberMeService($userRepository));

// Variabel output dipakai untuk menampilkan pesan login.
$output = "";
// Menandai apakah user baru selesai register.
$forceLoginPage = isset($_GET["registered"]);

// Jika datang dari halaman register, bersihkan login lama lalu tampilkan
pesan sukses.

```

```

if ($forceLoginPage) {
    $authService->logout();
    $session->start();
    $output = '<p style="color:green;">Registrasi berhasil. Silakan
login.</p>';
}

// Jika user sudah login, langsung arahkan ke halaman kelola peserta.
if (!$forceLoginPage && $authService->currentUser()) {
    Response::redirect("kelolapeserta.php");
}

// Jika belum dipaksa ke halaman login, coba pulihkan session dari cookie
remember me.
if (!$forceLoginPage) {
    $authService->restoreUserFromCookie();
}

// Jika login dari cookie berhasil, arahkan ke halaman peserta.
if (!$forceLoginPage && $authService->currentUser()) {
    Response::redirect("kelolapeserta.php");
}

// Proses login dijalankan saat form dikirim.
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    // Kirim data form login ke service auth.
    [$success, $message] = $authService->login(
        trim((string)($_POST["email"] ?? "")),
        (string)($_POST["password"] ?? ""),
        trim((string)($_POST["role"] ?? "")),
        isset($_POST["remember_me"])
    );

    // Jika login berhasil, arahkan ke halaman kelola peserta.
    if ($success) {
        Response::redirect("kelolapeserta.php");
    }

    // Simpan pesan error jika login gagal.
    $output = '<p style="color:red;">' . htmlspecialchars($message) . '</p>';
}
?>
<!DOCTYPE html>
<html lang="en">
<head>
<!-- Menentukan charset halaman. -->
<meta charset="UTF-8">
<!-- Membuat halaman responsif di berbagai ukuran layar. -->
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<!-- Judul tab browser. -->
<title>Login | InkluSkill</title>
<!-- Memanggil file CSS utama. -->
<link rel="stylesheet" href="styles.css">

```

```
</head>
<body class="auth-body">
<div class="auth-container">
<!-- Judul halaman login. -->
<h2 class="auth-title">Masuk ke InkluSkill</h2>

<!-- Form login user. -->
<form method="POST" action="" class="auth-form">
<div class="form-group">
<label>Email</label>
<input type="email" name="email" placeholder="Masukkan email" required>
</div>

<div class="form-group">
<!-- Input password user. -->
<label>Password</label>
<input type="password" name="password" placeholder="Masukkan password"
required>
</div>

<div class="form-group">
<!-- Pilihan role untuk login. -->
<label>Pilih Role</label>
<select name="role" required>
<option value="">-- Pilih Role --</option>
<option value="sekolah">Sekolah</option>
</select>
</div>

<div class="form-group">
<label>
<!-- Checkbox untuk remember me. -->
<input type="checkbox" name="remember_me" value="1">
Remember me
</label>
</div>

<!-- Tombol submit login. -->
<button class="btn auth-btn" type="submit">Masuk</button>
</form>

<!-- Menampilkan pesan login dari backend. -->
<?php echo $output; ?>

<p class="auth-switch">
<!-- Link menuju halaman daftar akun. -->
Belum punya akun?
<a href="register.php">Daftar</a>
</p>
</div>
</body>
</html>
```


INI ADALAH auth.php

```
/* =====
File CSS khusus halaman kelola peserta.
Bagian utama:
1. Reset dan font global
2. Layout sidebar dan konten utama
3. Tabel peserta
4. Tombol aksi
5. Modal tambah/edit peserta
===== */
@import
url("https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;500;600&display=swap");

/* Reset global + font default */
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
    font-family: Poppins, sans-serif;
}

/* Mengatur layout dasar halaman agar sidebar dan konten berdampingan. */
body {
    background: #f5f7fb;
    display: flex;
}

/* Styling sidebar navigasi di sisi kiri halaman. */
.sidebar {
    width: 240px;
    min-height: 100vh;
    background: white;
    padding: 20px;
    border-right: 1px solid #e4e4e4;
}

/* Styling judul brand atau nama aplikasi di sidebar. */
.sidebar-title {
    font-size: 20px;
    color: #4f46e5;
    margin-bottom: 25px;
    font-weight: 600;
}

/* Styling seluruh link menu yang ada di sidebar. */
.sidebar a {
    display: block;
    padding: 12px;
    border-radius: 10px;
    margin-bottom: 6px;
    text-decoration: none;
    color: #333;
```

```
    transition: 0.2s;
}

/* Warna menu saat hover dan saat link aktif. */
.sidebar a:hover,
.sidebar a.active {
    background: #e8efff;
    color: #4461d7;
}

/* Styling khusus tombol logout. */
.logout-btn {
    margin-top: 20px;
    width: 100%;
    padding: 12px;
    border: none;
    border-radius: 10px;
    background: #ff5b5b;
    color: white;
    cursor: pointer;
}

/* Area utama tempat header, search, tabel, dan modal. */
.main-content {
    flex: 1;
    padding: 30px;
}

/* Header atas pada area konten utama. */
header {
    display: flex;
    justify-content: space-between;
    margin-bottom: 25px;
}

/* Blok profil user yang tampil di kanan atas. */
.profile {
    display: flex;
    align-items: center;
    gap: 10px;
}

/* Styling gambar profil user. */
.profile img {
    width: 40px;
    height: 40px;
    border-radius: 50%;
}

/* Baris aksi berisi input pencarian dan tombol tambah peserta. */
.action-row {
    display: flex;
    justify-content: space-between;
```

```
    margin-bottom: 20px;
}

/* Styling input pencarian peserta. */
.action-row input {
    width: 260px;
    padding: 12px;
    border-radius: 10px;
    border: 1px solid #ddd;
}

/* Styling tombol tambah peserta. */
.add-btn {
    background: #4461d7;
    color: white;
    padding: 12px 20px;
    border: none;
    border-radius: 10px;
    cursor: pointer;
}

/* Kartu putih pembungkus tabel peserta. */
.table-card {
    background: white;
    padding: 20px;
    border-radius: 15px;
    box-shadow: 0 4px 15px rgba(0,0,0,0.05);
}

/* Styling dasar tabel peserta. */
table {
    width: 100%;
    border-collapse: collapse;
}

/* Styling kepala tabel. */
table th {
    padding: 12px;
    background: #eef2ff;
    text-align: left;
}

/* Styling isi tabel. */
table td {
    padding: 12px;
    border-bottom: 1px solid #eee;
}

/* Styling tombol edit. */
.edit {
    padding: 6px 10px;
    border: none;
    background: #e8efff;
}
```

```

        border-radius: 8px;
        color: #4461d7;
        cursor: pointer;
    }

/* Styling tombol hapus. */
.delete {
    padding: 6px 10px;
    border: none;
    background: #ffe1e1;
    border-radius: 8px;
    color: #ff4d4d;
    cursor: pointer;
}

/* Styling judul halaman kelola peserta. */
.title-page {
    font-size: 28px;
    font-weight: 600;
    display: flex;
    align-items: center;
    gap: 10px;
    color: #4f46e5;
}

/* Lapisan modal gelap untuk form tambah dan edit peserta. */
.modal {
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    background: rgba(0,0,0,0.35);
    display: none;
    justify-content: center;
    align-items: center;
}

/* Kotak isi modal. */
.modal-content {
    width: 380px;
    background: white;
    padding: 25px;
    border-radius: 15px;
    box-shadow: 0 4px 20px rgba(0,0,0,0.1);
}

/* Judul di dalam modal. */
.modal-content h3 {
    margin-bottom: 15px;
}

/* Styling semua input dan select dalam modal. */

```

```

.modal-content input,
.modal-content select {
    width: 100%;
    padding: 12px;
    margin-bottom: 12px;
    border-radius: 10px;
    border: 1px solid #ddd;
}

/* Baris tombol di bagian bawah modal. */
.modal-buttons {
    display: flex;
    justify-content: flex-end;
    gap: 10px;
}

/* Styling tombol simpan pada modal. */
#saveBtn {
    background: #4461d7;
    color: white;
    border: none;
    padding: 10px 18px;
    border-radius: 8px;
    cursor: pointer;
}

/* Styling tombol batal pada modal. */
#closeModal {
    background: #ddd;
    border: none;
    padding: 10px 18px;
    border-radius: 8px;
    cursor: pointer;
}

```

INI ADALAH kelolapeserta.css

```

// Mengambil elemen modal utama.
const modal = document.getElementById("modalForm");
// Tombol untuk membuka modal tambah peserta.
const openModal = document.getElementById("openModal");
// Tombol untuk menutup modal.
const closeModal = document.getElementById("closeModal");
// Form tambah dan edit peserta.
const pesertaForm = document.getElementById("pesertaForm");
// Judul modal.
const modalTitle = document.getElementById("modalTitle");
// Input hidden untuk membedakan aksi tambah atau edit.
const aksiInput = document.getElementById("aksiInput");
// Input hidden untuk id peserta saat edit.
const idInput = document.getElementById("idInput");
// Input hidden untuk menyimpan nama file foto lama.
const fotoLamaInput = document.getElementById("fotoLamaInput");
// Input nama peserta.

```

```

const namaInput = document.getElementById("namaInput");
// Input usia peserta.
const usiaInput = document.getElementById("usiaInput");
// Input jenis kelamin peserta.
const genderInput = document.getElementById("genderInput");
// Select pelatihan peserta.
const selectPelatihan = document.getElementById("inputPelatihan");
// Input file foto peserta.
const fotoInput = document.getElementById("fotoInput");
// Teks informasi foto saat ini.
const fotoInfo = document.getElementById("fotoInfo");
// Input pencarian peserta pada tabel.
const searchPeserta = document.getElementById("searchPeserta");

// Fungsi untuk memuat daftar pelatihan ke dropdown.
function loadPelatihanForPeserta(selectedValue = "") {
    // Ambil daftar pelatihan dari localStorage jika ada.
    const pelatihanList = JSON.parse(localStorage.getItem("pelatihanData")) ||
[];
    // Opsi fallback jika localStorage belum berisi data pelatihan.
    const fallbackOptions = ["Moshing", "Icikiwir"];
    // Kosongkan select sebelum diisi ulang.
    selectPelatihan.innerHTML = "";

    // Tentukan daftar opsi yang akan dipakai.
    const opsi = pelatihanList.length > 0
        ? pelatihanList.map((item) => item.nama ?? item.namaPelatihan ??
    "").filter(Boolean)
        : fallbackOptions;

    // Tambahkan semua opsi ke elemen select.
    opsi.forEach((namaPelatihan) => {
        const option = document.createElement("option");
        option.value = namaPelatihan;
        option.textContent = namaPelatihan;
        selectPelatihan.appendChild(option);
    });

    // Jika nilai lama tidak ada dalam opsi, tambahkan secara manual.
    if (selectedValue && !opsi.includes(selectedValue)) {
        const option = document.createElement("option");
        option.value = selectedValue;
        option.textContent = selectedValue;
        selectPelatihan.appendChild(option);
    }

    // Jika ada nilai terpilih, set value select ke nilai tersebut.
    if (selectedValue) {
        selectPelatihan.value = selectedValue;
    }
}

// Fungsi untuk mereset form ke mode tambah data.

```

```

function resetFormTambah() {
  // Reset seluruh field form.
  pesertaForm.reset();
  // Set aksi ke tambah.
  aksiInput.value = "tambah";
  // Kosongkan id peserta.
  idInput.value = "";
  // Kosongkan nama file foto lama.
  fotoLamaInput.value = "";
  // Ubah judul modal.
  modalTitle.textContent = "Tambah Peserta";
  // Saat tambah, foto wajib diisi.
  fotoInput.required = true;
  // Tampilkan info foto default.
  fotoInfo.textContent = "Belum ada foto dipilih.";
  // Set gender default.
  genderInput.value = "Laki-laki";
  // Muat ulang daftar pelatihan.
  loadPelatihanForPeserta();
}

// Event saat tombol tambah peserta diklik.
openModal.addEventListener("click", () => {
  // Reset form ke mode tambah.
  resetFormTambah();
  // Tampilkan modal.
  modal.style.display = "flex";
});

// Event saat tombol batal diklik.
closeModal.addEventListener("click", () => {
  // Sembunyikan modal.
  modal.style.display = "none";
});

// Menutup modal jika area gelap di luar modal diklik.
window.addEventListener("click", (event) => {
  if (event.target === modal) {
    modal.style.display = "none";
  }
});

// Fungsi untuk mengisi form ke mode edit berdasarkan data tombol yang diklik.
function editData(button) {
  // Ubah mode form menjadi edit.
  aksiInput.value = "edit";
  // Ambil id peserta dari data attribute tombol.
  idInput.value = button.dataset.id || "";
  // Ambil nama file foto lama dari data attribute tombol.
  fotoLamaInput.value = button.dataset.foto || "";
  // Isi nama peserta ke input.
  namaInput.value = button.dataset.nama || "";
  // Isi usia peserta ke input.

```

```

usiaInput.value = button.dataset.umur || "";
// Isi jenis kelamin peserta ke select.
genderInput.value = button.dataset.jk || "Laki-laki";
// Isi pelatihan sesuai data peserta.
loadPelatihanForPeserta(button.dataset.pelatihan || "");
// Kosongkan input file agar user bebas pilih file baru.
fotoInput.value = "";
// Saat edit, foto tidak wajib diganti.
fotoInput.required = false;
// Ganti judul modal ke edit.
modalTitle.textContent = "Edit Peserta";
// Tampilkan info foto lama jika ada.
fotoInfo.textContent = button.dataset.foto
    ? `Foto saat ini: ${button.dataset.foto}. Pilih file baru jika ingin
mengganti.`
    : "Belum ada foto. Pilih file untuk menambahkan foto.";
// Tampilkan modal edit.
modal.style.display = "flex";
}

// Daftarkan fungsi edit ke object window agar bisa dipanggil dari HTML
inline.
window.editData = editData;

// Event pencarian peserta pada tabel.
searchPeserta.addEventListener("keyup", function () {
    // Ambil kata kunci pencarian.
    const keyword = this.value.toLowerCase();
    // Ambil semua baris tabel peserta.
    const rows = document.querySelectorAll("#pesertaTable tr");

    // Loop setiap baris tabel.
    rows.forEach((row) => {
        // Ambil cell nama peserta.
        const namaCell = row.children[1];
        // Jika cell tidak ada, lewati baris ini.
        if (!namaCell) {
            return;
        }

        // Ambil teks nama peserta lalu ubah ke huruf kecil.
        const nama = namaCell.textContent.toLowerCase();
        // Tampilkan hanya baris yang cocok dengan kata kunci.
        row.style.display = nama.includes(keyword) ? "" : "none";
    });
});

// Muat daftar pelatihan saat halaman pertama kali dibuka.
loadPelatihanForPeserta();
// Tampilkan info awal untuk bagian foto.
fotoInfo.textContent = "Belum ada foto dipilih.";

```

INI ADALAH kelolapeserta.js


```

<?php

declare(strict_types=1);

// Memuat autoloader class OOP aplikasi.
require_once __DIR__ . "/app/bootstrap.php";

use App\Config\Database;
use App\Repositories\UserRepository;
use App\Services\AuthService;
use App\Services\ParticipantPhotoService;
use App\Services\ParticipantService;
use App\Services\RememberMeService;
use App\Support\Response;
use App\Support\SessionManager;

// Menyiapkan session manager untuk halaman peserta.
$session = new SessionManager();
// Memastikan session aktif.
$session->start();
// Menyiapkan repository untuk akses tabel user.
$userRepository = new UserRepository(Database::getConnection());
// Menyiapkan service auth untuk proteksi login.
$authService = new AuthService($session, $userRepository, new RememberMeService($userRepository));
// Menyiapkan service peserta untuk proses tambah, edit, hapus, dan tampil data.
$participantService = new ParticipantService(
    $userRepository,
    new ParticipantPhotoService(__DIR__ . DIRECTORY_SEPARATOR . "uploads")
);

// Pastikan halaman ini hanya bisa diakses oleh user yang sudah login.
$authService->requireLogin();

// Ambil flash message dari session lalu hapus setelah dibaca.
$pesan = (string)$session->pull("flash_pesan", "");

// Jalankan proses form saat request POST.
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    // Serahkan data form ke service peserta.
    [$success, $message, $shouldRedirect] = $participantService->handleForm($_POST, $_FILES);

    // Jika proses berhasil, simpan pesan ke session lalu redirect.
    if ($success && $shouldRedirect) {
        $session->set("flash_pesan", $message);
        Response::redirect("kelolapeserta.php");
    }

    // Jika gagal, tampilkan pesan error di halaman ini.
    $pesan = $message;
}

```

```

// Siapkan array data peserta untuk ditampilkan di tabel.
$daftarPeserta = [];
// Jika struktur tabel siap, ambil seluruh data peserta.
if ($participantService->canManageParticipants()) {
    $daftarPeserta = $participantService->getParticipants();
} elseif ($pesan === "") {
    // Tampilkan pesan jika struktur tabel belum lengkap.
    $pesan = "Tabel user belum siap ditampilkan. Pastikan ada kolom id,
nama/username, umur/usia, jenis_kelamin, pelatihan.";
}
?>
<!DOCTYPE html>
<html lang="id">
<head>
    <!-- Menentukan karakter encoding halaman. -->
    <meta charset="UTF-8" />
    <!-- Menyesuaikan tampilan agar responsif di mobile. -->
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <!-- Judul halaman browser. -->
    <title>Kelola Peserta</title>
    <!-- Memanggil stylesheet halaman peserta. -->
    <link rel="stylesheet" href="kelolapeserta.css" />
    <!-- Memanggil library icon Remix Icon. -->
    <link
href="https://cdn.jsdelivr.net/npm/remixicon@4.2.0/fonts/remixicon.css"
rel="stylesheet" />
</head>
<body>
    <!-- Jika ada pesan dari backend, tampilkan alert. -->
    <?php if ($pesan !== ""): ?>
        <!-- Menampilkan alert JavaScript. -->
        <script>alert('<?php echo addslashes($pesan); ?>');</script>
    <?php endif; ?>

    <!-- Sidebar menu aplikasi. -->
    <div class="sidebar">
        <!-- Judul sidebar. -->
        <h2 class="sidebar-title">InkluSkill</h2>
        <!-- Link dashboard. -->
        <a href="dinas.html"><i class="ri-dashboard-line"></i> Dashboard</a>
        <!-- Link halaman aktif kelola peserta. -->
        <a class="active" href="#"><i class="ri-user-3-line"></i> Kelola
Peserta</a>
        <!-- Link halaman pelatihan. -->
        <a href="peltihan.html"><i class="ri-calendar-check-line"></i>
Pelatihan & Jadwal</a>
        <!-- Link halaman kehadiran. -->
        <a href="kehadiran.html"><i class="ri-time-line"></i> Kehadiran</a>
        <!-- Link halaman statistik. -->
        <a href="statistik.html"><i class="ri-bar-chart-box-line"></i>
Statistik</a>
        <!-- Link halaman laporan. -->

```

```

        <a href="laporan.html"><i class="ri-file-list-3-line"></i> Laporan</a>
        <!-- Tombol logout. -->
        <a class="logout-btn" href="logout.php"><i class="ri-logout-box-
line"></i> Logout</a>
    </div>

    <!-- Konten utama halaman. -->
    <div class="main-content">
        <!-- Bagian header konten. -->
        <header>
            <!-- Judul halaman konten. -->
            <h1 class="title-page"><i class="ri-user-3-line"></i> Kelola
Peserta</h1>
            <!-- Profil pengguna login. -->
            <div class="profile">
                <!-- Gambar admin. -->
                
                <!-- Menampilkan nama user dari session. -->
                <span><?php echo htmlspecialchars($authService->username() ?
"Dinas Sosial"); ?></span>
            </div>
        </header>

        <!-- Baris aksi pencarian dan tombol tambah. -->
        <div class="action-row">
            <!-- Input pencarian peserta. -->
            <input type="text" id="searchPeserta" placeholder="Cari
peserta...">
            <!-- Tombol membuka modal tambah peserta. -->
            <button class="add-btn" id="openModal" type="button"><i class="ri-
add-circle-line"></i> Tambah Peserta</button>
        </div>

        <!-- Card tabel peserta. -->
        <div class="table-card">
            <!-- Tabel data peserta. -->
            <table>
                <!-- Bagian kepala tabel. -->
                <thead>
                    <!-- Baris judul kolom. -->
                    <tr>
                        <!-- Kolom nomor. -->
                        <th>No</th>
                        <!-- Kolom nama peserta. -->
                        <th>Nama Peserta</th>
                        <!-- Kolom usia peserta. -->
                        <th>Usia</th>
                        <!-- Kolom jenis kelamin. -->
                        <th>Jenis Kelamin</th>
                        <!-- Kolom pelatihan. -->
                        <th>Pelatihan Diikuti</th>
                        <!-- Kolom foto. -->
                        <th>Foto</th>

```

```

        <!-- Kolom aksi. -->
        <th>Aksi</th>
    </tr>
</thead>
<!-- Bagian body tabel. -->
<tbody id="pesertaTable">
    <!-- Jika data peserta tersedia. -->
    <?php if (!empty($daftarPeserta)): ?>
        <!-- Inisialisasi nomor urut. -->
        <?php $no = 1; ?>
        <!-- Loop seluruh data peserta. -->
        <?php foreach ($daftarPeserta as $data): ?>
            <!-- Susun URL foto jika ada file foto. -->
            <?php $fotoUrl = !empty($data["foto"]) ?
"uploads/" . rawurlencode($data["foto"]) : ""; ?>
            <!-- Satu baris data peserta. -->
            <tr>
                <!-- Menampilkan nomor urut. -->
                <td><?php echo $no++; ?></td>
                <!-- Menampilkan nama peserta. -->
                <td><?php echo
htmlspecialchars($data["nama"]); ?></td>
                <!-- Menampilkan umur peserta. -->
                <td><?php echo
htmlspecialchars((string)$data["umur"]); ?></td>
                <!-- Menampilkan jenis kelamin peserta. -->
                <td><?php echo
htmlspecialchars($data["jenis_kelamin"]); ?></td>
                <!-- Menampilkan pelatihan peserta. -->
                <td><?php echo
htmlspecialchars($data["pelatihan"]); ?></td>
                <!-- Menampilkan kolom foto. -->
                <td>
                    <!-- Jika foto tersedia. -->
                    <?php if ($fotoUrl != ""): ?>
                        <!-- Tampilkan gambar foto peserta. --
                    >
                        
                    <?php else: ?>
                        <!-- Tampilkan teks jika tidak ada
foto. -->
                        Tidak ada foto
                    <?php endif; ?>
                </td>
                <!-- Kolom aksi edit dan hapus. -->
                <td>
                    <!-- Tombol edit data peserta. -->
                    <button
                        class="edit"
                        type="button"

```

```

data-id="<?php echo (int)$data["id"];
?>"
data-nama="<?php echo
htmlspecialchars($data["nama"], ENT_QUOTES); ?>"
data-umur="<?php echo
htmlspecialchars((string)$data["umur"], ENT_QUOTES); ?>"
data-jk="<?php echo
htmlspecialchars($data["jenis_kelamin"], ENT_QUOTES); ?>"
data-pelatihan="<?php echo
htmlspecialchars($data["pelatihan"], ENT_QUOTES); ?>"
data-foto="<?php echo
htmlspecialchars($data["foto"] ?? "", ENT_QUOTES); ?>"
onclick="editData(this)"
>Edit</button>
<!-- Form hapus data peserta. -->
<form method="POST"
style="display:inline;" onsubmit="return confirm('Hapus peserta ini?');">
<!-- Menandai aksi hapus. -->
<input type="hidden" name="aksi"
value="hapus">
<!-- Menyimpan id peserta. -->
<input type="hidden" name="id"
value="<?php echo (int)$data["id"]; ?>">
<!-- Menyimpan nama file foto lama. --
>
<input type="hidden" name="foto_lama"
value="<?php echo htmlspecialchars($data["foto"] ?? "", ENT_QUOTES); ?>">
<!-- Tombol submit hapus. -->
<button class="delete"
type="submit">Hapus</button>
</form>
</td>
</tr>
<?php endforeach; ?>
<?php else: ?>
<!-- Jika data kosong, tampilkan pesan data belum ada.
-->
<tr><td colspan="7">Data belum ada</td></tr>
<?php endif; ?>
</tbody>
</table>
</div>
</div>

<!-- Modal form tambah dan edit peserta. -->
<div class="modal" id="modalForm">
<!-- Kontainer isi modal. -->
<div class="modal-content">
<!-- Judul modal. -->
<h3 id="modalTitle">Tambah Peserta</h3>
<!-- Form tambah/edit peserta. -->
<form action="" method="POST" enctype="multipart/form-data"
id="pesertaForm">

```

```

        <!-- Menyimpan jenis aksi tambah atau edit. -->
        <input type="hidden" name="aksi" id="aksiInput"
value="tambah">
        <!-- Menyimpan id peserta saat edit. -->
        <input type="hidden" name="id" id="idInput" value="">
        <!-- Menyimpan nama file foto lama saat edit. -->
        <input type="hidden" name="foto_lama" id="fotoLamaInput"
value="">

        <!-- Label nama peserta. -->
        <label>Nama Peserta</label>
        <!-- Input nama peserta. -->
        <input type="text" name="nama" id="namaInput" required>

        <!-- Label usia peserta. -->
        <label>Usia</label>
        <!-- Input usia peserta. -->
        <input type="number" name="umur" id="usiaInput" required>

        <!-- Label jenis kelamin peserta. -->
        <label>Jenis Kelamin</label>
        <!-- Pilihan jenis kelamin. -->
        <select id="genderInput" name="jenis_kelamin" required>
            <!-- Opsi laki-laki. -->
            <option value="Laki-laki">Laki-laki</option>
            <!-- Opsi perempuan. -->
            <option value="Perempuan">Perempuan</option>
        </select>

        <!-- Label pelatihan peserta. -->
        <label>Pelatihan</label>
        <!-- Pilihan pelatihan. -->
        <select id="inputPelatihan" name="pelatihan" required>
            <!-- Opsi pelatihan pertama. -->
            <option value="Moshing">Moshing</option>
            <!-- Opsi pelatihan kedua. -->
            <option value="Icikiwir">Icikiwir</option>
        </select>

        <!-- Label upload foto. -->
        <label>Upload Foto</label>
        <!-- Input file untuk foto peserta. -->
        <input type="file" name="foto" id="fotoInput"
accept=".jpg,.jpeg,.png,.gif,.webp">
        <!-- Info kecil tentang foto saat ini. -->
        <p id="fotoInfo" style="margin:8px 0 0; font-size:14px;
color:#555;"></p>

        <!-- Tombol aksi modal. -->
        <div class="modal-buttons">
            <!-- Tombol simpan form. -->
            <button id="saveBtn" type="submit">Simpan</button>
            <!-- Tombol batal menutup modal. -->

```

```

        <button id="closeModal" type="button">Batal</button>
    </div>
</form>
</div>
</div>

<!-- Memanggil file JavaScript halaman peserta. -->
<script src="kelolapeserta.js"></script>
</body>
</html>

```

INI adalah kelolapeserta.php

```

<?php

declare(strict_types=1);

// Memuat autoloader class OOP aplikasi.
require_once __DIR__ . "/app/bootstrap.php";

use App\Config\Database;

// Menyimpan koneksi database agar tetap bisa dipakai oleh file lama yang
membutuhkan variabel $koneksi.
$koneksi = Database::getConnection();

```

INI ADALAH koneksi.php

```

<?php

declare(strict_types=1);

// Memuat autoloader class OOP aplikasi.
require_once __DIR__ . "/app/bootstrap.php";

use App\Config\Database;
use App\Repositories\UserRepository;
use App\Services\AuthService;
use App\Services\RememberMeService;
use App\Support\Response;
use App\Support\SessionManager;

// Menyiapkan session manager.
$session = new SessionManager();
// Memastikan session aktif.
$session->start();
// Menyiapkan repository user.
$userRepository = new UserRepository(Database::getConnection());
// Menyiapkan service auth untuk menghapus session login.
$authService = new AuthService($session, $userRepository, new
RememberMeService($userRepository));

// Menjalankan proses logout.
$authService->logout();

```

```
// Mengarahkan user kembali ke halaman login.  
Response::redirect("auth.php");
```

INI ADALAH logout.php

```
<?php  
  
declare(strict_types=1);  
  
// Memuat autoloader class OOP aplikasi.  
require_once __DIR__ . "/app/bootstrap.php";  
  
use App\Config\Database;  
use App\Repositories\UserRepository;  
use App\Services\AuthService;  
use App\Services\RememberMeService;  
use App\Support\Response;  
use App\Support\SessionManager;  
  
// Menyiapkan session manager untuk kebutuhan halaman register.  
$session = new SessionManager();  
// Memastikan session aktif.  
$session->start();  
// Menyiapkan repository untuk akses data user.  
$userRepository = new UserRepository(Database::getConnection());  
// Menyiapkan service auth yang menangani proses register.  
$authService = new AuthService($session, $userRepository, new  
RememberMeService($userRepository));  
  
// Variabel pesan dipakai untuk menampilkan error ke pengguna.  
$pesan = "";  
  
// Jalankan proses register hanya saat form dikirim dengan method POST.  
if ($_SERVER["REQUEST_METHOD"] === "POST") {  
    // Kirim data form ke service auth.  
    [$success, $message] = $authService->register(  
        trim((string)$_POST["username"] ?? ""),  
        trim((string)$_POST["email"] ?? ""),  
        (string)$_POST["password"] ?? "",  
        trim((string)$_POST["role"] ?? "")  
    );  
  
    // Jika register berhasil, arahkan user ke halaman login.  
    if ($success) {  
        Response::redirect("auth.php?registered=1");  
    }  
  
    // Simpan pesan error jika register gagal.  
    $pesan = $message;  
}  
?  
  
<!DOCTYPE html>  
<html lang="en">
```



```
<head>

<meta charset="UTF-8">

<!-- agar halaman responsive di mobile -->
<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Register | InkluSkill</title>

<!-- memanggil file CSS -->
<link rel="stylesheet" href="styles.css">

</head>

<body class="auth-body">
    <?php
        // Tampilkan notifikasi jika ada pesan dari proses register.
        if ($pesan !== "") {
            echo '<script>alert("'" . addslashes($pesan) . "');</script>';
        }
    ?>

<div class="auth-container">

<!-- Judul halaman register -->
<h2 class="auth-title">Buat Akun Baru</h2>

<!-- FORM REGISTER -->
<form method="POST" class="auth-form">

<!-- INPUT NAMA -->
<div class="form-group">

<label>Nama Lengkap</label>

<input type="text" name="username" placeholder="Masukkan nama" required>

</div>

<!-- INPUT EMAIL -->
<div class="form-group">

<label>Email</label>

<input type="email" name="email" placeholder="Masukkan email" required>

</div>
```

```

<!-- INPUT PASSWORD -->
<div class="form-group">

<label>Password</label>

<input type="password" name="password" placeholder="Buat password" required>

</div>

<!-- PILIH ROLE -->
<div class="form-group">

<label>Pilih Role</label>

<select name="role">

<option value="">-- Pilih Role --</option>

<option value="sekolah">Sekolah</option>

</select>

</div>

<!-- TOMBOL DAFTAR -->
<button class="btn auth-btn" type="submit">

Daftar

</button>

</form>

<!-- LINK KE HALAMAN LOGIN -->
<p class="auth-switch">

Sudah punya akun?

<a href="auth.php">Masuk</a>

</p>

</div>

</body>
</html>

```

INI ADALAH register.php

```

<?php

declare(strict_types=1);

// Memuat autoloader class OOP aplikasi.
require_once __DIR__ . "/app/bootstrap.php";

use App\Config\Database;
use App\Repositories\UserRepository;
use App\Services\AuthService;
use App\Services\RememberMeService;
use App\Support\SessionManager;

// Helper untuk membuat object AuthService sekali panggil.
function buildAuthService(): AuthService
{
    // Menyiapkan session manager.
    $session = new SessionManager();
    // Memastikan session aktif.
    $session->start();
    // Menyiapkan repository user untuk kebutuhan autentikasi.
    $repository = new UserRepository(Database::getConnection());

    // Mengembalikan object AuthService yang siap digunakan.
    return new AuthService($session, $repository, new
RememberMeService($repository));
}

// Helper untuk mencoba memulihkan session user dari cookie remember me.
function restoreRememberedUser(mysqli $koneksi): bool
{
    // Parameter $koneksi tidak dipakai karena koneksi diambil langsung dari
class Database.
    unset($koneksi);

    return buildAuthService()->restoreUserFromCookie();
}

// Helper untuk proses logout user.
function logoutUser(): void
{
    buildAuthService()->logout();
}
?>

```

INI ADALAH session_helper.php

```

/* =====
styles.css
File CSS global aplikasi.
Digunakan oleh landing page, halaman auth, dan beberapa bagian dashboard.
Struktur utama:
1) Variabel & reset
2) Komponen umum

```

```

3) Section landing page
4) Auth page
5) Dashboard
6) Footer
7) Responsive breakpoints
===== */
@import
url("https://fonts.googleapis.com/css2?family=Poppins:ital,wght@0,100;0,200;0,300;0,400;0,500;0,600;0,700;0,800;0,900;1,100;1,200;1,300;1,400;1,500;1,600;1,700;1,800;1,900&display=swap");

/* Variabel warna, lebar maksimal, dan token global untuk konsistensi desain.
*/
:root {
  --primary-color: #524fa1;
  --primary-color-dark: #3b397a;
  --text-dark: #262626;
  --text-light: #737373;
  --extra-light: #e5e5e5;
  --white: #ffffff;
  --max-width: 1200px;
}

/* Reset default browser agar tampilan lebih seragam di semua device. */
* {
  padding: 0;
  margin: 0;
  box-sizing: border-box;
}

/* Kontainer umum untuk setiap section besar di halaman. */
.section__container {
  max-width: var(--max-width);
  margin: auto;
  padding: 5rem 1rem;
}

/* Judul utama section. */
.section__header {
  max-width: 900px;
  margin-inline: auto;
  margin-bottom: 1rem;
  font-size: 2.5rem;
  font-weight: 700;
  color: var(--text-dark);
  text-align: center;
}

/* Warna highlight untuk teks tertentu di judul section. */
.section__header span {
  color: var(--primary-color);
}

```

```
/* Deskripsi pendek di bawah judul section. */
.section__description {
  max-width: 600px;
  margin-inline: auto;
  color: var(--text-light);
  line-height: 1.75rem;
  text-align: center;
}

/* Tombol utama yang dipakai di beberapa halaman. */
.btn {
  padding: 1rem 2rem;
  outline: none;
  border: none;
  font-size: 1rem;
  color: var(--white);
  background-color: var(--primary-color);
  border-radius: 5px;
  transition: 0.3s;
  cursor: pointer;
}

/* Efek hover tombol umum. */
.btn:hover {
  background-color: var(--primary-color-dark);
  box-shadow: 2px 2px 10px rgba(106, 56, 194, 0.5);
}

/* Logo utama aplikasi. */
.logo {
  font-size: 1.5rem;
  font-weight: 700;
  color: var(--primary-color);
}

/* Warna aksen tambahan pada bagian span logo. */
.logo span {
  color: #fa6021;
}

/* Aturan global untuk elemen gambar. */
img {
  display: flex;
  width: 100%;
}

/* Aturan global untuk link. */
a {
  text-decoration: none;
  transition: 0.3s;
}

/* Menghilangkan bullet bawaan list. */
```

```
ul {
  list-style: none;
}

/* Pengaturan scroll halus untuk perpindahan anchor. */
html,
body {
  scroll-behavior: smooth;
}

/* Font default aplikasi. */
body {
  font-family: "Poppins", sans-serif;
}

/* Navigasi utama pada halaman landing page. */
nav {
  position: fixed;
  isolation: isolate;
  width: 100%;
  max-width: var(--max-width);
  margin-inline: auto;
  z-index: 9;
}

/* Header kecil pada area nav versi mobile. */
.nav__header {
  padding: 1rem;
  width: 100%;
  display: flex;
  align-items: center;
  justify-content: space-between;
  background-color: var(--extra-light);
}

/* Tombol menu hamburger mobile. */
.nav__menu__btn {
  font-size: 1.5rem;
  color: var(--text-dark);
  cursor: pointer;
}

/* Daftar link navigasi. */
.nav__links {
  position: absolute;
  top: 65px;
  left: 0;
  width: 100%;
  display: flex;
  align-items: center;
  justify-content: center;
  flex-direction: column;
  gap: 2rem;
}
```

```

padding: 2rem;
background-color: var(--extra-light);
transition: 0.5s;
z-index: -1;
transform: translateY(-100%);
}

/* Kondisi menu saat dibuka di mobile. */
.nav__links.open {
  transform: translateY(0);
}

/* Styling link di dalam nav. */
.nav__links a {
  font-weight: 500;
  color: var(--text-dark);
}

/* Warna link saat hover. */
.nav__links a:hover {
  color: var(--primary-color);
}

/* Section hero atau pembuka landing page. */
.hero__container {
  position: relative;
  isolation: isolate;
  overflow: hidden;
}

/* Badge kecil di atas judul hero. */
.hero__container h2 {
  max-width: fit-content;
  margin-inline: auto;
  margin-bottom: 1rem;
  padding: 0.5rem 1rem;
  display: flex;
  align-items: center;
  gap: 10px;
  font-size: 0.9rem;
  font-weight: 600;
  color: #f15a22;
  background-color: #fff8f5;
  border-radius: 5rem;
}

/* Gambar kecil di dalam badge hero. */
.hero__container h2 img {
  max-width: 25px;
}

/* Judul utama hero section. */
.hero__container h1 {

```

```
margin-bottom: 1rem;
font-size: 4rem;
font-weight: 700;
color: var(--text-dark);
text-align: center;
line-height: 5.5rem;
}

/* Warna aksen pada sebagian teks judul hero. */
.hero__container h1 span {
  color: var(--primary-color);
}

/* Deskripsi hero section. */
.hero__container p {
  margin-bottom: 2rem;
  max-width: 600px;
  margin-inline: auto;
  color: var(--text-light);
  line-height: 2rem;
  text-align: center;
}

/* Kontainer tombol-tombol hero. */
.hero__btns {
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 2rem;
}

/* Link pendukung di area hero. */
.hero__btns a {
  display: flex;
  align-items: center;
  gap: 10px;
  font-size: 1rem;
  font-weight: 500;
  color: var(--text-dark);
}

/* Lingkaran ikon pada link hero. */
.hero__btns a span {
  padding: 5px 11px;
  font-size: 1.5rem;
  color: var(--white);
  background-color: var(--primary-color);
  border-radius: 100%;
  transition: 0.3s;
}

/* Efek hover ikon pada hero. */
.hero__btns a:hover span {
```



```
    box-shadow: 2px 2px 10px rgba(106, 56, 194, 0.5);
}

/* Background section workflow atau langkah penggunaan. */
.workflow {
    background-image: url("assets/steps-bg.png");
    background-position: center center;
    background-size: cover;
    background-repeat: no-repeat;
}

/* Grid kartu workflow. */
.workflow__grid {
    margin-top: 4rem;
    display: grid;
    gap: 1rem;
}

/* Styling kartu workflow. */
.workflow__card {
    padding: 1rem;
    background-color: var(--white);
    border-radius: 5px;
    box-shadow: 5px 5px 20px rgba(0, 0, 0, 0.1);
}

/* Ikon utama pada kartu workflow. */
.workflow__card span {
    display: inline-block;
    margin-bottom: 1rem;
    padding: 5px 11px;
    font-size: 1.5rem;
    border-radius: 100%;
}

/* Warna ikon kartu workflow pertama. */
.workflow__card:nth-child(1) span {
    color: #fa4e09;
    background-color: #fff9f6;
}

/* Warna ikon kartu workflow kedua. */
.workflow__card:nth-child(2) span {
    color: #6a38c2;
    background-color: #e9ddff;
}

/* Warna ikon kartu workflow ketiga. */
.workflow__card:nth-child(3) span {
    color: #3ac2ba;
    background-color: #f0fffe;
}
```

```
/* Warna ikon kartu workflow keempat. */
.workflow__card:nth-child(4) span {
  color: #fbbc09;
  background-color: #fff8e3;
}

/* Judul pada kartu workflow. */
.workflow__card h4 {
  margin-bottom: 0.5rem;
  font-size: 1.2rem;
  font-weight: 700;
  color: var(--text-dark);
}

/* Deskripsi pada kartu workflow. */
.workflow__card p {
  color: var(--text-light);
}

/* Grid daftar pelatihan atau layanan utama. */
.training__grid {
  margin-block: 4rem;
  display: grid;
  gap: 1rem;
}

/* Kartu pelatihan individual. */
.training__card {
  padding: 1rem;
  border-radius: 5px;
  box-shadow: 5px 5px 20px rgba(0, 0, 0, 0.1);
  transition: 0.3s;
}

/* Efek hover kartu pelatihan. */
.training__card:hover {
  background-color: var(--primary-color);
}

/* Ikon pada kartu pelatihan. */
.training__card span {
  display: inline-block;
  margin-bottom: 1rem;
  padding: 5px 11px;
  font-size: 1.5rem;
  border-radius: 5px;
  transition: 0.3s;
}

/* Warna kartu pelatihan pertama. */
.training__card:nth-child(1) span {
  color: #f04a0c;
  background-color: #f6efef;
}
```

```
}

/* Warna kartu pelatihan kedua. */
.training__card:nth-child(2) span {
  color: #6a38c2;
  background-color: #e9ddff;
}

/* Warna kartu pelatihan ketiga. */
.training__card:nth-child(3) span {
  color: #ff0101;
  background-color: #fff2f2;
}

/* Warna kartu pelatihan keempat. */
.training__card:nth-child(4) span {
  color: #fbbc09;
  background-color: #fff8e3;
}

/* Warna kartu pelatihan kelima. */
.training__card:nth-child(5) span {
  color: #4680e7;
  background-color: #e7edf8;
}

/* Warna kartu pelatihan keenam. */
.training__card:nth-child(6) span {
  color: #34a753;
  background-color: #f1fef5;
}

/* Warna kartu pelatihan ketujuh. */
.training__card:nth-child(7) span {
  color: #443ee0;
  background-color: #f6f5ff;
}

/* Warna kartu pelatihan kedelapan. */
.training__card:nth-child(8) span {
  color: #3ac2ba;
  background-color: #f0fffe;
}

/* Warna ikon saat kartu pelatihan dihover. */
.training__card:hover span {
  color: var(--white);
  background-color: #794cc7;
}

/* Judul kartu pelatihan. */
.training__card h4 {
  margin-bottom: 0.5rem;
}
```

```
font-size: 1.2rem;
font-weight: 700;
color: var(--text-dark);
transition: 0.3s;
}

/* Deskripsi kartu pelatihan. */
.training__card p {
  color: var(--text-light);
  transition: 0.3s;
}

/* Warna judul saat kartu dihover. */
.training__card:hover h4 {
  color: var(--white);
}

/* Warna deskripsi saat kartu dihover. */
.training__card:hover p {
  color: var(--extra-light);
}

/* Grid statistik pada landing page atau dashboard. */
.statistics__grid {
  margin-top: 4rem;
  display: grid;
  gap: 2rem;
}

/* Kartu statistik individual. */
.statistics__card {
  padding: 2rem;
  text-align: center;
  border-radius: 10px;
  background-color: var(--white);
  box-shadow: 5px 5px 20px rgba(0, 0, 0, 0.1);
  transition: 0.3s;
}

/* Efek hover kartu statistik. */
.statistics__card:hover {
  transform: translateY(-10px);
  box-shadow: 5px 15px 30px rgba(106, 56, 194, 0.2);
}

/* Lingkaran ikon statistik. */
.statistics__icon {
  display: inline-block;
  margin-bottom: 1rem;
  padding: 1rem;
  font-size: 2.5rem;
  border-radius: 50%;
  transition: 0.3s;
}
```

```

}

/* Warna ikon statistik pertama. */
.statistics__card:nth-child(1) .statistics__icon {
  color: #6a38c2;
  background-color: #e9ddff;
}

/* Warna ikon statistik kedua. */
.statistics__card:nth-child(2) .statistics__icon {
  color: #f04a0c;
  background-color: #fff9f6;
}

/* Warna ikon statistik ketiga. */
.statistics__card:nth-child(3) .statistics__icon {
  color: #3ac2ba;
  background-color: #f0fffe;
}

/* Angka besar statistik. */
.statistics__number {
  margin-bottom: 1rem;
  font-size: 3.5rem;
  font-weight: 700;
  color: var(--primary-color);
  line-height: 1;
}

/* Teks deskripsi statistik. */
.statistics__card p {
  color: var(--text-light);
  line-height: 1.5rem;
  font-size: 0.95rem;
}

/* Grid daftar services atau fitur utama. */
.services__grid {
  margin-top: 4rem;
  display: grid;
  gap: 2rem 1rem;
}

/* Gambar pada kartu services. */
.services__card img {
  margin-bottom: 1rem;
  border-radius: 5px;
}

/* Detail teks di samping garis aksan service. */
.services__details {
  display: flex;
  align-items: flex-start;
}

```

```

}

/* Garis aksan pada sisi kiri teks service. */
.services__details div {
  padding-left: 1rem;
  border-left: 2px solid var(--primary-color);
}

/* Judul service. */
.services__details h4 {
  font-size: 1.2rem;
  font-weight: 700;
  color: var(--text-dark);
}

/* Deskripsi service. */
.services__details p {
  color: var(--text-light);
}

/* Container utama swiper testimonial. */
.swiper {
  padding-top: 4rem;
  padding-bottom: 4rem;
  width: 100%;
  max-width: 600px;
}

/* Foto profil pada kartu testimonial. */
.testimonial__card img {
  max-width: 80px;
  margin-inline: auto;
  margin-bottom: 2rem;
  border-radius: 100%;
  box-shadow: 2px 2px 10px rgba(0, 0, 0, 0.1);
}

/* Teks isi testimonial. */
.testimonial__card p {
  margin-bottom: 1rem;
  line-height: 1.75rem;
  color: var(--text-dark);
  text-align: center;
}

/* Baris bintang rating testimonial. */
.testimonial__ratings {
  margin-bottom: 1rem;
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 5px;
}

```

```
/* Warna icon bintang rating. */
.testimonial__ratings span {
  color: goldenrod;
}

/* Nama pemberi testimonial. */
.testimonial__card h4 {
  font-size: 1.2rem;
  font-weight: 600;
  color: var(--text-dark);
  text-align: center;
}

/* Posisi atau peran pemberi testimonial. */
.testimonial__card h5 {
  font-size: 1rem;
  font-weight: 500;
  color: var(--text-light);
  text-align: center;
}

/* Tombol navigasi swiper kanan dan kiri. */
.swiper-button-next,
.swiper-button-prev {
  color: var(--primary-color);
  width: 35px;
  height: 35px;
  background-color: var(--white);
  border-radius: 50%;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
  z-index: 20 !important;
}

/* Ukuran ikon di tombol swiper. */
.swiper-button-next:after,
.swiper-button-prev:after {
  font-size: 18px;
  font-weight: 700;
}

/* Posisi tombol next. */
.swiper-button-next {
  right: -15px;
}

/* Posisi tombol previous. */
.swiper-button-prev {
  left: -15px;
}

/* Efek hover untuk tombol navigasi swiper. */
.swiper-button-next:hover,
```

```
.swiper-button-prev:hover {
  background-color: var(--primary-color-dark);
  color: var(--primary-color);
}

/* Warna bullet aktif pada pagination swiper. */
.swiper-pagination-bullet-active {
  background-color: var(--primary-color);
}

/* Layout halaman auth: login dan register. */
.auth-body {
  background: var(--extra-light);
  min-height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
}

/* Kotak utama form auth. */
.auth-container {
  width: 100%;
  max-width: 400px;
  padding: 2rem;
  background: var(--white);
  box-shadow: 5px 5px 20px rgba(0,0,0,0.08);
  border-radius: 10px;
}

/* Judul halaman auth. */
.auth-title {
  text-align: center;
  font-size: 2rem;
  font-weight: 700;
  color: var(--primary-color);
  margin-bottom: 2rem;
}

/* Jarak antar grup input pada auth form. */
.auth-form .form-group {
  margin-bottom: 1.2rem;
}

/* Label pada input auth. */
.auth-form label {
  display: block;
  font-weight: 600;
  margin-bottom: 5px;
  color: var(--text-dark);
}

/* Input dan select pada halaman auth. */
.auth-form input,
```



```
.auth-form select {
  width: 100%;
  padding: 0.8rem;
  border-radius: 6px;
  border: 1px solid var(--extra-light);
  background: #fafafa;
  outline: none;
  font-size: 1rem;
}

/* Tombol auth memenuhi lebar kontainer. */
.auth-btn {
  width: 100%;
  margin-top: 1rem;
}

/* Area teks switch login/register. */
.auth-switch {
  margin-top: 1rem;
  text-align: center;
}

/* Link pada teks switch auth. */
.auth-switch a {
  color: var(--primary-color);
}

/* Bar navigasi dashboard sederhana. */
.dash-nav {
  width: 100%;
  padding: 1.5rem;
  background: var(--extra-light);
  display: flex;
  justify-content: space-between;
  align-items: center;
}

/* Tombol logout umum dashboard. */
.logout-btn {
  padding: 0.6rem 1.3rem;
  background: var(--primary-color);
  color: var(--white);
  border-radius: 6px;
}

/* Kontainer utama dashboard. */
.dashboard-container {
  max-width: var(--max-width);
  margin: auto;
  padding: 3rem 1rem;
}

/* Grid kartu dashboard. */
```

```
.dashboard-grid {
  margin-top: 3rem;
  display: grid;
  gap: 1.5rem;
}

/* Kartu ringkasan dashboard. */
.dash-card {
  padding: 2rem;
  background: var(--white);
  box-shadow: 5px 5px 20px rgba(0,0,0,0.08);
  border-radius: 10px;
  text-align: center;
}

/* Judul kecil di kartu dashboard. */
.dash-card h3 {
  margin-bottom: 0.5rem;
  color: var(--primary-color);
}

/* Penyesuaian posisi tombol swiper untuk layar lebih besar. */
@media (width > 768px) {
  .swiper-button-next {
    right: -50px;
  }

  .swiper-button-prev {
    left: -50px;
  }
}

/* Grid utama footer. */
.footer__container {
  display: grid;
  gap: 4rem 2rem;
}

/* Logo footer. */
.footer__logo {
  margin-bottom: 2rem;
}

/* Paragraf pada kolom footer. */
.footer__col p {
  max-width: 400px;
  color: var(--text-light);
}

/* Judul setiap kolom footer. */
.footer__col h4 {
  margin-bottom: 2rem;
  font-size: 1.2rem;
}
```

```
font-weight: 600;
color: var(--text-dark);
}

/* Daftar link di footer. */
.footer__links {
  display: grid;
  gap: 1rem;
}

/* Link di footer. */
.footer__links a {
  color: var(--text-light);
}

/* Warna link footer saat hover. */
.footer__links a:hover {
  color: var(--primary-color);
}

/* Ukuran icon pada link footer. */
.footer__links a span {
  font-size: 1.25rem;
}

/* Bar copyright paling bawah. */
.footer__bar {
  padding: 1rem;
  font-size: 0.9rem;
  color: var(--text-light);
  text-align: center;
}

/* Breakpoint untuk mobile besar. */
@media (width > 540px) {
  .workflow__grid {
    grid-template-columns: repeat(2, 1fr);
  }

  .training__grid {
    grid-template-columns: repeat(2, 1fr);
  }

  .statistics__grid {
    grid-template-columns: repeat(2, 1fr);
  }

  .services__grid {
    grid-template-columns: repeat(2, 1fr);
  }

  .footer__container {
    grid-template-columns: repeat(2, 1fr);
  }
}
```

```
}  
}  
  
/* Breakpoint untuk tablet. */  
@media (width > 768px) {  
  nav {  
    position: static;  
    padding: 2rem 1rem;  
    display: flex;  
    align-items: center;  
    justify-content: space-between;  
    gap: 2rem;  
  }  
  
  .nav__header {  
    padding: 0;  
    background-color: transparent;  
  }  
  
  .nav__menu__btn {  
    display: none;  
  }  
  
  .nav__links {  
    position: static;  
    padding: 0;  
    flex-direction: row;  
    justify-content: flex-end;  
    background-color: transparent;  
    transform: none;  
  }  
  
  .workflow__grid {  
    margin-top: 6rem;  
    grid-template-columns: repeat(4, 1fr);  
  }  
  
  .workflow__card:nth-child(2n - 1) {  
    transform: translateY(-2rem);  
  }  
  
  .training__grid {  
    grid-template-columns: repeat(4, 1fr);  
  }  
  
  .statistics__grid {  
    grid-template-columns: repeat(3, 1fr);  
  }  
  
  .services__grid {  
    grid-template-columns: repeat(3, 1fr);  
  }  
}
```

```

.footer__container {
  grid-template-columns: repeat(5, 1fr);
}

.footer__col:nth-child(1) {
  grid-column: 1/3;
}
}

/* Breakpoint untuk desktop besar. */
@media (width > 1024px) {
  .workflow__card {
    padding: 1.5rem;
  }

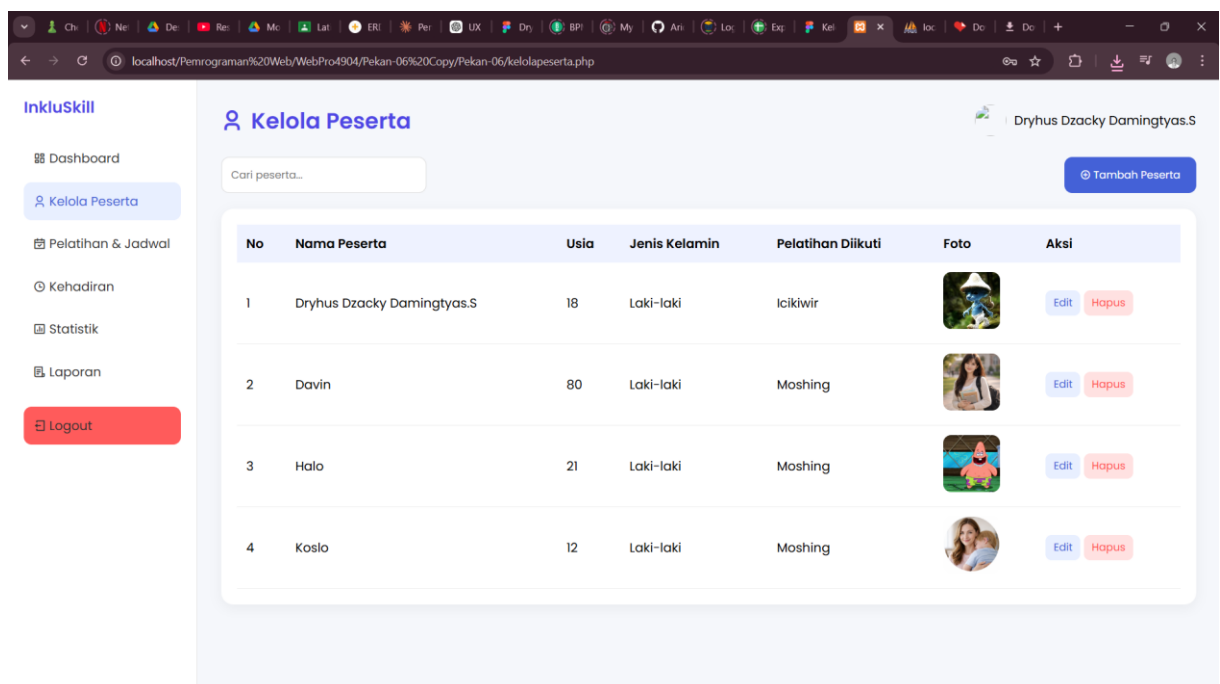
  .training__card {
    padding: 1.5rem;
  }

  .services__grid {
    gap: 2rem;
  }

  .statistics__card {
    padding: 2.5rem;
  }
}

```

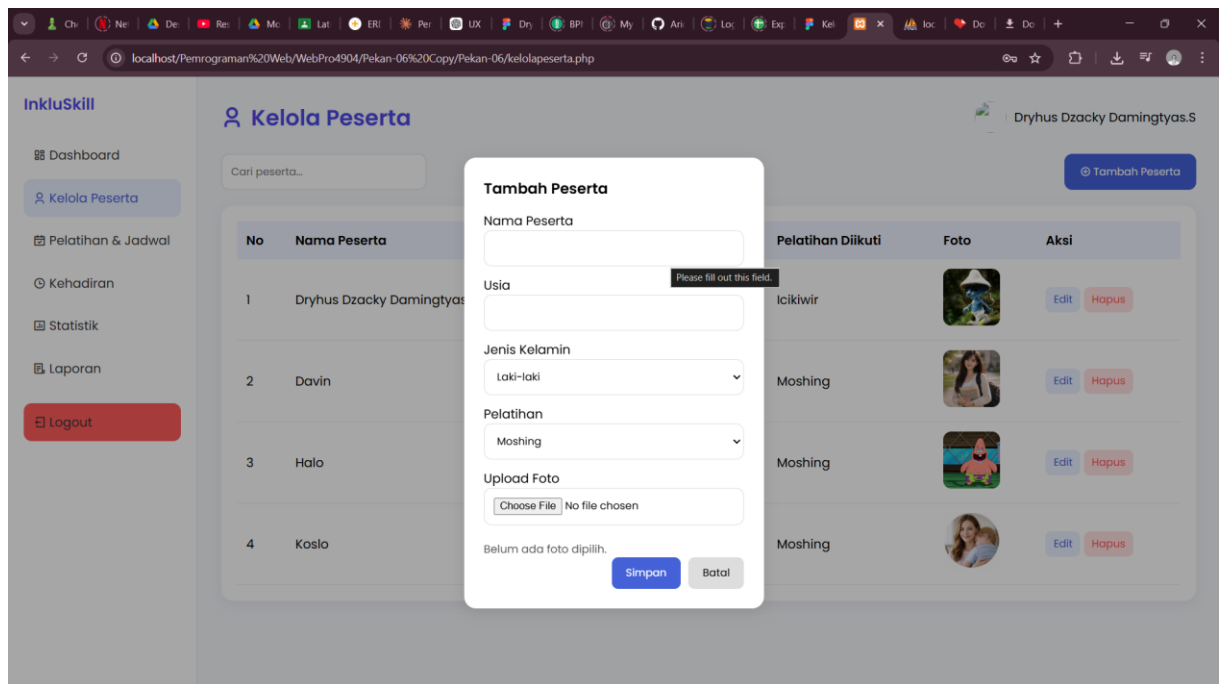
Ini adalah styles.css



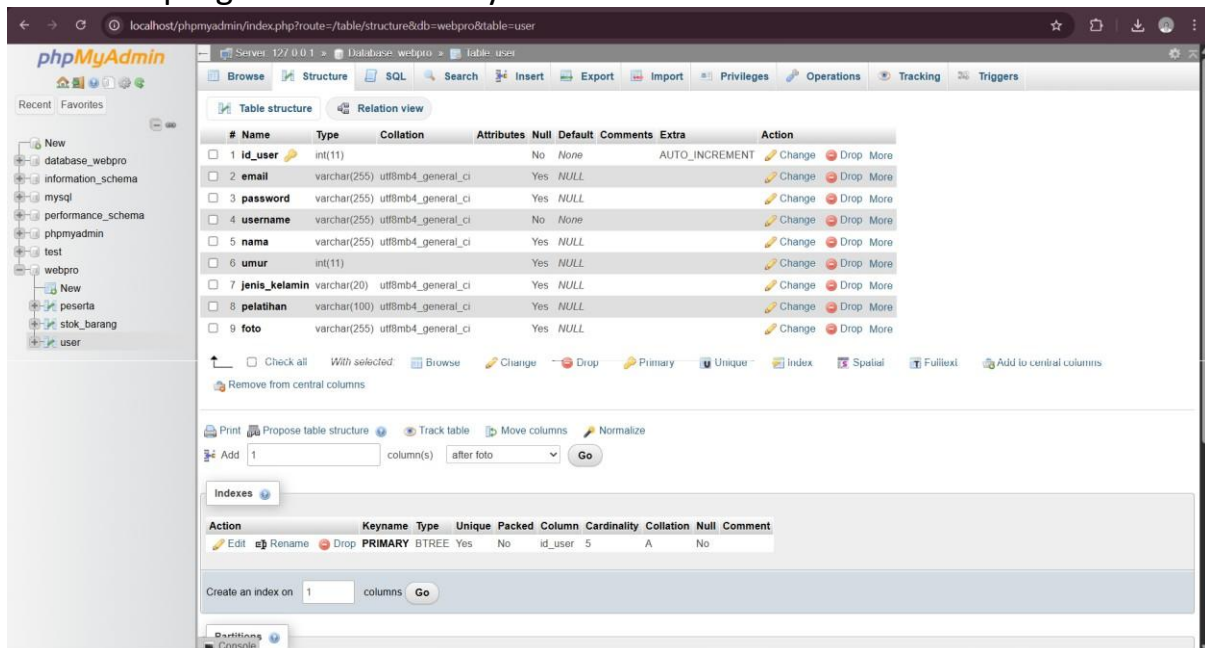
The screenshot shows a web application interface for 'InklusSkill'. The main section is titled 'Kelola Peserta' (Manage Participants). It features a search bar, a 'Tambah Peserta' (Add Participant) button, and a table listing participants.

No	Nama Peserta	Usia	Jenis Kelamin	Pelatihan Diikuti	Foto	Aksi
1	Dryhus Dzacky Damingtyas.S	18	Laki-laki	Icikiwir		Edit Hapus
2	Davin	80	Laki-laki	Moshing		Edit Hapus
3	Halo	21	Laki-laki	Moshing		Edit Hapus
4	Koslo	12	Laki-laki	Moshing		Edit Hapus

ini adalah hasil output program



Ini adalah program edit CRUD nya



Ini adalaah struktur database nya

Showing rows 0 - 9 (10 total, Query took 0.0004 seconds)

```
SELECT * FROM `user`
```

Number of rows: 25 Filter rows: Search this table Sort by key: None

	id_user	email	password	username	nama	umur	jenis_kelamin	pelatihan	foto
☐ Edit ☐ Copy ☐ Delete	20	dzacky98ku@gmail.com	938b3b4406bdfc1aaddf17e801dd1c	Dryhus Dzacky Damingtyas S	NULL	NULL	NULL	NULL	NULL
☐ Edit ☐ Copy ☐ Delete	24	davingay@gmail.com	cb9e6df0cbb653c13181472b3a68f3b	Davin Maw	NULL	NULL	NULL	NULL	NULL
☐ Edit ☐ Copy ☐ Delete	28	dryhus@gmail.com	811470f08c3caf5cc8113de188b66207	Dryhus Ganteng	NULL	NULL	NULL	NULL	NULL
☐ Edit ☐ Copy ☐ Delete	30	dryhus@gmail.com	811470f08c3caf5cc8113de188b66207	Dryhus	NULL	NULL	NULL	NULL	NULL
☐ Edit ☐ Copy ☐ Delete	31	NULL	NULL	Dryhus Dzacky Damingtyas S	18	Laki-laki	lckiwir	peserta_69cdf960046dd9.6262	
☐ Edit ☐ Copy ☐ Delete	32	davinickwir@gmail.com	8b900377fc9a207bfc21c0e4ee88cf	Davin lckiwir	NULL	NULL	NULL	NULL	NULL
☐ Edit ☐ Copy ☐ Delete	33	NULL	NULL	Davin	80	Laki-laki	Moshing	peserta_69ce0f05729d2.2262	
☐ Edit ☐ Copy ☐ Delete	34	halo@gmail.com	07d8222114abb6999a2b54c512e4ab9e	Halo	NULL	NULL	NULL	NULL	NULL
☐ Edit ☐ Copy ☐ Delete	35	NULL	NULL	Halo	21	Laki-laki	Moshing	peserta_69d67f32d15474.4462	
☐ Edit ☐ Copy ☐ Delete	37	NULL	NULL	Koslo	12	Laki-laki	Moshing	peserta_69d6881b89e2f4.6762	

Number of rows: 25 Filter rows: Search this table Sort by key: None

Ini adalah isi database nya

E. Analisis dan Pembahasan

Pada praktikum ini, program dikembangkan dengan menerapkan konsep Object-Oriented Programming (OOP) pada aplikasi web berbasis PHP. Penerapan OOP dilakukan dengan memisahkan fungsi program ke dalam class-class yang memiliki tanggung jawab masing-masing. Dengan cara ini, kode menjadi lebih terstruktur, lebih mudah dipahami, dan lebih mudah dikembangkan dibandingkan pendekatan prosedural.

Pada program ini, class Database berfungsi sebagai penyedia koneksi database. Class ini memastikan bahwa koneksi ke database dapat digunakan oleh bagian lain program secara terpusat. Dengan adanya pemisahan ini, perubahan konfigurasi database cukup dilakukan pada satu tempat saja. Hal ini menunjukkan salah satu keuntungan OOP, yaitu memudahkan pengelolaan kode.

Selanjutnya, class UserRepository digunakan untuk menangani seluruh proses yang berhubungan dengan data user di database. Class ini bertugas mengambil data user, menambahkan akun baru, mengecek struktur kolom tabel, serta mengelola data peserta. Penggunaan repository membuat logika akses data terpisah dari halaman tampilan, sehingga file seperti register.php, auth.php, dan kelolapeserta.php tidak langsung berisi query SQL yang terlalu banyak.

Class AuthService digunakan untuk menangani proses autentikasi, seperti login, register, logout, dan validasi user. Dari hasil praktikum, class ini membantu menyederhanakan alur proses login karena validasi dan pengelolaan session tidak ditulis berulang-ulang di setiap halaman. Selain itu, fitur remember me juga dipisahkan ke dalam RememberMeService, sehingga fungsi penyimpanan cookie dan pemulihan login menjadi lebih rapi.

Pada fitur kelola peserta, digunakan class ParticipantService dan ParticipantPhotoService. ParticipantService bertugas memproses data peserta seperti tambah, edit, dan hapus, sedangkan ParticipantPhotoService khusus menangani upload dan penghapusan file foto. Pemisahan ini menunjukkan bahwa dalam OOP, setiap class sebaiknya memiliki satu tanggung jawab utama. Dengan begitu, jika ada perubahan pada fitur upload foto, perbaikan cukup dilakukan pada class terkait tanpa memengaruhi seluruh program.

Selain itu, class SessionManager dan Response digunakan sebagai pendukung untuk pengelolaan session dan redirect halaman. Meskipun sederhana, class-class ini membantu membuat kode lebih konsisten dan lebih mudah digunakan kembali. Hal ini membuktikan bahwa OOP tidak hanya dipakai pada proses besar seperti login dan CRUD, tetapi juga bisa diterapkan pada bagian utilitas program.

Dari hasil pengujian, program dapat menjalankan beberapa fitur utama dengan baik, yaitu registrasi akun, login user, penyimpanan session, logout, penambahan peserta, pengeditan peserta, penghapusan peserta, serta upload foto peserta. Perbaikan juga dilakukan pada tampilan nama user di pojok kanan atas, sehingga sistem menampilkan nama hasil input saat registrasi, bukan email. Ini menunjukkan bahwa data session dan pengambilan data user dari database sudah berjalan sesuai kebutuhan.

Secara keseluruhan, penerapan OOP pada praktikum ini memberikan beberapa manfaat utama, yaitu:

- **kode program menjadi lebih rapi dan terorganisir,**
- **tiap bagian program memiliki tanggung jawab yang jelas,**
- **kode lebih mudah diperbaiki dan dikembangkan,**
- **pengulangan kode dapat dikurangi,**
- **hubungan antarbagian program menjadi lebih mudah dipahami.**

Namun demikian, penerapan OOP juga membuat struktur program menjadi lebih banyak file dan class. Bagi pemula, hal ini bisa terasa lebih rumit dibandingkan pemrograman prosedural. Meskipun begitu, untuk aplikasi yang terus berkembang, pendekatan OOP lebih baik karena mempermudah perawatan program dalam jangka panjang.

F. Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa penerapan Object-Oriented Programming (OOP) pada aplikasi web berbasis PHP berhasil diterapkan dengan baik. Konsep OOP membuat program menjadi lebih terstruktur karena setiap bagian program dibagi ke dalam class sesuai tugas dan fungsinya masing-masing. Dengan menggunakan OOP, proses pengelolaan koneksi database, autentikasi user, pengelolaan session, serta fitur CRUD data peserta menjadi lebih rapi dan mudah dipahami. Selain itu, pemisahan logika program ke dalam beberapa class juga mempermudah pengembangan, perbaikan, dan pemeliharaan program.

Hasil praktikum menunjukkan bahwa fitur-fitur utama seperti registrasi, login, logout, pengelolaan data peserta, dan upload foto dapat berjalan dengan baik. Penerapan OOP juga membantu mengurangi penulisan kode yang berulang dan membuat alur program lebih jelas.

Dengan demikian, praktikum ini membuktikan bahwa OOP sangat bermanfaat dalam pembuatan aplikasi web, terutama untuk membangun program yang lebih modular, efisien, dan mudah dikembangkan di kemudian hari.

<https://github.com/AriqAhmadNayaka/WebPro4904/tree/DryhusDzackyDamingtyas.S-707012530010>